Python Learning Notes (Lessons 1–12 + Toolkit)

Quick-revision notes for the Python covered while practising (`main.py` , `toolkit.py` , and the mini-projects — see `Exercises.md` for the full list). Each lesson has: the concept, examples, and the **gotchas** we actually hit.

> **Setup reminder:** run a file with `python3 main.py` . This machine uses **Python 3.11**. `#` starts a comment (Python ignores the rest of the line). See also `AI_ROADMAP.md` for where this is all heading.

Table of Contents

1. Variables & `print()`
2. Strings & Numbers
3. Conditionals (`if` / `elif` / `else`)
4. Loops (`for` / `while`)
5. Lists
6. Functions
7. Dictionaries
8. Error handling (`try` / `except`)
9. Slicing & String Methods
10. Sets & the Dict-Counter Pattern
11. List Comprehensions
12. Tuples & Unpacking - Toolkit extras (picked up while problem-solving) - Design principles (learned from real bugs) - Style conventions (PEP 8) - Common error types cheat sheet

---

Lesson 1 — Variables & `print()`

A **variable** is a name holding a value. Assign with `=` . No type declaration needed — Python infers it.

```
name = "Raghav"      # string (text)
age = 30             # int (whole number)
height = 5.9         # float (decimal)
is_learning = True   # bool (True / False)

print(name)                   # Raghav
print("Age is:", age)         # Age is: 30   (print takes multiple comma-separated items)
```

- Strings need quotes (`"hi"` or `'hi'`). Numbers don't. `"30"` is text; `30` is a number.
- Variable names must match **exactly**, character for character (a typo = a different / non-existent name).

Lesson 2 — Strings & Numbers

Arithmetic operators

```
a, b = 10, 3
a + b    # 13   addition
a - b    # 7    subtraction
a * b    # 30   multiplication
a / b    # 3.333...  division (ALWAYS gives a float)
a // b   # 3    floor division (drops remainder)
a % b    # 1    modulo (the remainder)  ← "n % 2 == 0" tests even
a ** b   # 1000 power
```

f-strings (the modern way to build text)

Put `f` before the quote, drop variables/expressions inside `{ }`:

```
name, age = "Raj", 25
print(f"My name is {name} and I am {age} years old")
print(f"Next year I'll be {age + 1}")   # math works inside the braces
```

No need for `+` inside an f-string — just place each variable in its own `{ }`.

String concatenation (the older way)

```
full = first + " " + last     # gluing strings with +
```

Type conversion (casting)

```
int("25")       # 25      text  → integer
float("3.14")   # 3.14    text  → float
str(25)         # "25"    number → text
```

⚠️ **Gotchas** - `"2" + "3"` → `"23"` (concatenation!), but `2 + 3` → `5`. Convert first: `int("2") + 3` → `5`. - `int("3.5")` → **ValueError** (`int()` won't parse a decimal *string*). `int(3.5)` → `3` (truncates a real float).

Lesson 3 — Conditionals

Comparisons → produce `True` / `False`

```
x == 10   # equal to      (⚠️ == compares; = assigns — classic beginner bug)
x != 3    # not equal
x > 5   x < 5   x >= 10   x <= 10
90 > score > 80           # chained comparison (Python-specific, handy!)
```

`if` / `elif` / `else`

```
if score >= 90:
    print("A")
elif score >= 80:      # only checked if the ones above were False
    print("B")
elif score >= 70:
    print("C")
else:
    print("Fail")
```

- Condition line ends with a **colon** `:`.
- Body is **indented** (4 spaces). Indentation IS the syntax — it defines what's "inside".
- `elif` runs only the **first** matching branch, top to bottom.

Combine conditions

```
if age >= 18 and has_ticket:   # both must be True
if age < 13 or age > 65:       # at least one True
if not is_member:              # flips True/False
```

⚠️ **Gotcha — boundary bugs (`>` vs `>=`):** "18 or older" means `age >= 18`, not `age > 18`. Whenever a rule says "at least / or older / up to / minimum", check whether the boundary value itself should count.

Style: no parentheses around conditions in Python → `if x > 5:` not `if (x > 5):`.

Lesson 4 — Loops

`for` loop with `range()`

```
for i in range(5):        # 0,1,2,3,4  (starts at 0, STOPS BEFORE 5)
    print(i)

range(1, 6)      # 1,2,3,4,5      (start, stop)
range(0, 10, 2)  # 0,2,4,6,8      (start, stop, step)
```

Loop over a string

```
for letter in "cat":      # c, a, t
    print(letter)
```

`while` loop — runs while a condition stays True

```
count = 5
while count > 0:
    print(count)
    count -= 1            # MUST move toward making the condition False
print("Liftoff!")         # not indented → runs ONCE after the loop
```

`while True` + `break` — loop until you decide to stop

The go-to pattern for menus/games (unknown number of rounds):

```
while True:                    # loops forever...
    choice = input("Option: ")
    if choice == "quit":
        break                  # ...until break exits the loop right now
```

`break`, `continue`, `pass` — loop/flow control keywords

```
for n in nums:
    if n == target:
        break        # exit the loop immediately
    if n < 0:
        continue     # skip to the NEXT iteration (don't run rest of body)
# pass = "do nothing" placeholder (valid where syntax needs a statement)
if condition:
    pass             # (does nothing — a no-op)
```

⚠️ **Which is which:** `break` = leave the loop · `continue` = skip to next lap (loops only) · `pass` = do nothing (anywhere) · `return` = exit the whole function (functions only). Using `continue` outside a loop is a **SyntaxError**.

⚠️ **Gotchas** - `range(1, 11)` to include 10 — the stop value is **excluded**. - A `while` with nothing changing the condition = **infinite loop** (Ctrl+C to stop). - Shortcuts: `count += 1`, `count -= 1`, `count *= 2`, etc. - Indentation decides whether a line is inside or after the loop. - **Off-by-one:** `range(0, 7)` runs **7** times (0–6), not 6. Setting a flag like `done = True` does NOT stop the current iteration — use `break` to leave immediately.

Lesson 5 — Lists

Ordered collection of values in one variable. Uses square brackets `[ ]`.

```
fruits = ["apple", "banana", "cherry"]
fruits[0]    # apple    (indexing starts at 0)
fruits[-1]   # cherry   (negative counts from the end; -1 = last, self-adjusting)
len(fruits)  # 3

fruits[0] = "avocado"      # replace
fruits.append("date")      # add to end   (a "method" — function on the list)
fruits.remove("banana")    # remove by VALUE (first match)
fruits.pop(0)              # remove by INDEX, and returns the removed item
fruits.pop()              # no arg = remove & return the LAST item

if "apple" in fruits:      # membership check
    print("have apples")
```

⚠️ **`.pop(i)` = by position · `.remove(v)` = by value.** Pick the one matching what you know. ⚠️ **`.pop(0)` when the list is empty → IndexError.** And `.pop(-1)` is valid (removes last!), so a "0 means invalid" input can silently delete the last item — bounds-check first.

Looping over a list (most common loop in Python)

```
for fruit in fruits:
    print(f"I like {fruit}")     # no index/range/len needed
```

⚠️ **Values vs indices:** `for x in fruits` gives you the **values**; `for i in range(len(fruits))` gives you the **indices** (use `fruits[i]` for the value). Mixing these up is a classic bug. ⚠️ **Don't remove items from a list while looping it by index** — the indices shift underneath you. Build a new (filtered) list instead, e.g. with a comprehension.

⚠️ Aliasing — assigning a list does NOT copy it

```
a = [1, 2, 3]
b = a            # b is NOT a copy — both names point to the SAME list
b.pop(0)
print(a)         # [2, 3]  ← a changed too! 😱
```

To make a real, independent copy:

```
b = a.copy()     # the clear way
b = a[:]         # slice-of-everything also copies
b = list(a)      # so does list(...)
```

Same applies to dicts and sets. This is one of Python's top gotchas — mutating one alias changes them all. (This bit us in Mastermind; the fix in Tic-Tac-Toe's board display was `.copy()`.)

⚠️ **More gotchas** - `fruits[3]` when only 3 items exist → **IndexError** (valid: 0,1,2). - Prefer `fruits[-1]` over a hardcoded index for "the last item".

Lesson 6 — Functions

A reusable, named block of code. Define once with `def`, then **call** it.

```
def greet(name):              # `name` is a PARAMETER
    return f"Hello, {name}!"   # returns a value to the caller

msg = greet("Raj")            # "Raj" is the ARGUMENT
print(msg)                    # Hello, Raj!
```

- Defining ≠ running. Code runs only when you **call** it: `greet("Raj")`.
- Colon + indented body, as always.

`return` vs `print` (key distinction)

- `print` → shows text on screen, hands back nothing usable.
- `return` → gives a value back so the rest of the program can store/reuse it.
- Once a function hits `return`, it **exits immediately** (later lines don't run).

Guard clause (clean pattern for bad input)

```
def greet(name, language):
    if language not in languages:
        return "Invalid language"     # handle bad case & bail early
    # main logic sits un-indented below
    ...
```

`None`

A bare `return` (or reaching the end with no `return`) hands back the special value **`None`** ("no value"). Keep a function's return type **consistent** — don't sometimes return a string and sometimes `None`. Check with `if x is None:` (use `is`, not `==`, with None).

Handy built-ins for lists: `sum(nums)`, `len(nums)` → e.g. `average = sum(nums) / len(nums)`.

⚠️ **Gotcha:** `average([])` → **ZeroDivisionError** (empty list, `len` is 0). Guard against empty input.

Lesson 7 — Dictionaries

Collection of **key** → **value** pairs. Uses curly braces `{ }`. Access by *key*, not position.

```
person = {"name": "Raj", "age": 25}
person["name"]                  # Raj
person["age"] = 26              # change existing value
person["city"] = "Bangalore"    # add a new key (just assign)

person.get("email")               # None  (safe — no crash if missing)
person.get("email", "no email")   # default if missing
"name" in person                  # True — key exists?
len(person)                       # number of pairs
```

Looping (use `.items()` to get key AND value)

```
for key, value in person.items():    # unpacks each pair into two variables
    print(f"{key}: {value}")
```

`person.keys()`, `person.values()`, `person.items()` give the keys / values / pairs.

List of dictionaries (very common — looks like JSON/API data)

```
students = [
    {"name": "Raj", "grade": 40},
    {"name": "Anita", "grade": 60},
]
for s in students:
    print(f"{s['name']}: {s['grade']}")
```

⚠️ **Gotchas** - `person["missing"]` → **KeyError**. Use `.get()` to avoid crashing. - **f-string quote nesting (Python 3.11):** don't reuse the same quote inside `{ }`. Rule: **outer double `"`, inner single `'`** → `f"{book['title']}"`. (Allowed in

3.12+, but we're on 3.11.) - `{x}` on its own (no `f`, not inside a string) creates a **set**, not an f-string. `print("Avg", {avg})` prints `{43.3}` — a set! Use `print(f"Avg {avg}")` or `print("Avg", avg)`.

Lesson 8 — Error Handling

Some errors are unavoidable at runtime (bad user input, missing key, divide by zero). Handle them gracefully instead of crashing.

`input()` + conversion

```
name = input("Name? ")            # pauses, reads text; ALWAYS returns a string
age = int(input("Age? "))         # read text, then convert to int (read inside-out)
```

⚠️ `input()` always returns a **string** — `input()` + math without converting → **TypeError**.

`try` / `except`

```
try:
    number = int(input("Enter a number: "))
    print(number * 2)
except ValueError:                # runs only if a ValueError occurred
    print("That wasn't a valid number!")
```

- `try` block runs normally; if a line raises an error, Python **jumps** to the matching `except` and skips the rest of `try`.
- No error → `except` is skipped.

Multiple / specific handlers

```
try:
    ...
except ValueError:
    print("Bad value")
except ZeroDivisionError:
    print("Divided by zero")
# else:  runs if NO error   |   finally:  always runs (cleanup)
```

How the right branch is chosen: the failing *operation* decides the error's **type** (e.g. `10/0` → `ZeroDivisionError`). Python checks `except` blocks top-to-bottom and runs the **first matching type**. If none match, the program crashes with that error (by design — only catch what you expect).

⚠️ **Don't** use a bare `except:` that swallows everything — it hides real bugs. Catch the specific type.

Lesson 9 — Slicing & String Methods

Slicing — grab a *range*: `s[start:stop]` (stop EXCLUDED, like `range`)

```
s = "python"        # p=0 y=1 t=2 h=3 o=4 n=5
s[0:3]   # "pyt"
s[:3]    # "pyt"     (omit start = from the beginning)
s[3:]    # "hon"     (omit stop = to the end)
s[-3:]   # "hon"     (last 3)
s[:-1]   # "pytho"   (all but the last)
s[1:-1]  # "ytho"    (drop first AND last)
s[::2]   # "pto"     (step of 2 = every 2nd char)
s[::-1]  # "nohtyp"  ← REVERSE (memorize this one)
```

Works **identically on lists**: `nums[1:3]`, `nums[::-1]` (reversed list).

Essential string methods

```
"  Hi  ".strip()           # "Hi"       (trim surrounding whitespace)
"Hi".lower() / .upper()    # "hi" / "HI"
"Hello".replace("l", "L")  # "HeLLo"
"a,b,c".split(",")         # ["a","b","c"]     (string → list, cut on separator)
"hello world".split()      # ["hello","world"] (no arg = split on whitespace)
"-".join(["a","b","c"])    # "a-b-c"           (list → string, glued by separator)
"hello".find("l")          # 2  (index of first match, -1 if absent)
"hello".count("l")         # 2
len("hello")               # 5  (works on strings AND lists)
"policy.".strip(".,!?")    # "policy"  (strip() takes chars to trim off the ENDS)
```

`.split()` and `.join()` are inverses — split a sentence into words, process, join back.

Validation methods — return `True`/`False` for a string

```
"5".isdigit()      # True  — all characters are digits
"abc".isalpha()    # True  — all characters are letters
"abc123".isalnum() # True  — all letters and/or digits (no spaces/punctuation)
"HI".isupper()     # True   |   "hi".islower()  # True
```

Great for input checks: `if len(guess) == 1 and guess.isalpha(): ...` (used in Hangman, the password checker, Mastermind).

⚠️ **Strings are immutable.** Methods return a **new** string; they don't change the original. You MUST reassign: `s = s.upper()` — not just `s.upper()`. (`s[0] = "P"` is an error.) This is the #1 trap: calling `.strip()` and wondering why nothing changed.

Palindrome check (real LeetCode-style): `word == word[::-1]` → returns `True`/`False` directly (return the comparison; don't write `if ...: return True else: return False`).

Lesson 10 — Sets & the Dict-Counter Pattern

Sets — no duplicates, instant lookup

```
nums = [1, 2, 2, 3, 3, 3]
set(nums)            # {1, 2, 3}  → duplicates removed automatically
seen = set()
seen.add(5)          # add an item (adding again does nothing — no dupes)
5 in seen            # True  ← INSTANT (O(1)), no matter how big
```

- Created with `set()` or `{ }` **with items** (careful: empty `{}` is a *dict*, not a set).
- **No order, no duplicates.** Use when you only care *whether* you've seen something.
- Contrast: `x in my_list` must **scan** the whole list (O(n)). Set/dict lookup is instant.
- Remove duplicates: `set(items)`. Detect duplicates: `len(items) != len(set(items))`.

Dict-counter pattern — count occurrences (THE most common pattern)

```
words = ["apple", "banana", "apple"]
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1   # .get(w, 0) = current count, or 0 if unseen
# {'apple': 2, 'banana': 1}
```

The `.get(key, 0) + 1` one-liner replaces the manual `if key in counts: ... else: ...`. Answers: most frequent element, does a letter repeat, are two strings anagrams, etc.

Hash-map lookup pattern (from LeetCode "Two Sum")

Turn a slow **search** into an instant **lookup** by remembering what you've seen in a dict:

```
seen = {}                          # value -> index
for i in range(len(nums)):
    complement = target - nums[i]
    if complement in seen:         # instant check — have we seen the needed number?
        return [seen[complement], i]
    seen[nums[i]] = i              # remember this number + where it was
```

One pass = **O(n)**, vs nested loops = **O(n²)**. This is *the* key Easy/Medium trick.

Lesson 11 — List Comprehensions

A concise way to **build a list in one line** — replaces "empty list + loop + append".

The long way → the comprehension

```
# long way:
squares = []
for n in range(5):
    squares.append(n * n)

# comprehension (same result: [0, 1, 4, 9, 16]):
squares = [n * n for n in range(5)]
```

Read left-to-right: **`n * n` — for each `n` in `range(5)`.** (Expression first, then the loop.)

With a filter (`if` at the end)

```
evens = [n for n in range(10) if n % 2 == 0]      # [0, 2, 4, 6, 8]
```

Read: "`n` — for each n in range(10) — *if* n is even."

Works on any iterable (strings, lists, list-of-dicts)

```
words = ["hello", "hi", "hey"]
lengths = [len(w) for w in words]                 # [5, 2, 3]
upper   = [w.upper() for w in words]              # ['HELLO', 'HI', 'HEY']
short   = [w for w in words if len(w) <= 2]       # ['hi']

# over a dataset (list of dicts) — filter rows, extract a field:
movies = [{"title": "Inception", "rating": 8.8}, {"title": "Whiplash", "rating": 8.5}]
good = [m["title"] for m in movies if m["rating"] >= 8.7]   # ['Inception']
ratings = [m["rating"] for m in movies]
avg = sum(ratings) / len(ratings)                 # aggregate after extracting
```

When to use: simple transform and/or filter. If logic gets complex (many steps, nested conditions), a normal `for` loop is clearer — don't cram everything into one line.

⚠️ **Gotcha:** `>` vs `>=` in the filter is the same boundary trap as Lesson 3 — "7 or higher" is `>= 7`, not `> 7`.

Lesson 12 — Tuples & Unpacking

Tuple — like a list, but fixed (immutable), written with `( )`

```
point = (3, 5)
point[0]            # 3   (index like a list)
len(point)          # 2
# point[0] = 9      # 💥 TypeError — tuples can't be changed
```

Use for a fixed grouping that shouldn't change: coordinates `(x, y)`, RGB `(255, 0, 0)`, a record. Lists = collections you grow/shrink; tuples = fixed groupings.

Unpacking — pull values into separate variables in one line

```
point = (3, 5)
x, y = point            # x = 3, y = 5
a, b = 1, 2             # assign two at once
a, b = b, a             # swap — no temp variable needed
```

You already use this: `for key, value in person.items():` and `for i, item in enumerate(...)` work because each loop hands you a tuple that gets unpacked.

Functions can return multiple values (bundled in a tuple)

```
def min_max(nums):
    return min(nums), max(nums)     # returns a tuple (min, max)

low, high = min_max([4, 8, 2, 9])   # unpacked directly → low=2, high=9
```

This is *the* standard ML pattern, e.g. `train_data, test_data = split(data)`.

List of tuples — loop by unpacking

```
results = [("Raj", 85), ("Anita", 92)]
for name, score in results:         # each tuple unpacked into name, score
    print(f"{name} scored {score}")
```

Toolkit Extras

Small but important things picked up while building projects & solving problems.

Modules & `import` (the standard library)

```
import random
random.randint(1, 100)            # random whole number 1..100 (both ends included)
random.choice(["rock", "paper"])  # randomly pick one item from a list
```

`import` at the top of the file unlocks ready-made functionality. `random` powered the guessing game and RPS. (Many more modules exist: `math`, `datetime`, `json`, ...)

`time` — measure elapsed time

```
import time
start = time.time()      # current time in seconds (a big float)
# ... user does something ...
elapsed = time.time() - start    # seconds passed (used in the typing-speed test)
```

Handy built-in functions (no import needed)

```
sum([1, 2, 3])          # 6
min([4, 2, 9])          # 2       max([4, 2, 9])   # 9
len("hi") / len([1,2])  # 2 / 2   (length of string or list)
sorted([3, 1, 2])       # [1, 2, 3]  (returns a NEW sorted list)
abs(-5)                 # 5
all([True, True, False])# False   — True only if EVERY item is truthy
any([False, True])      # True    — True if AT LEAST ONE item is truthy
```

`all(...)` / `any(...)` shine with a comprehension inside — e.g. the Tic-Tac-Toe win check: `all(cell in my_marks for cell in winning_line)` → "did I fill this whole line?"

`Counter` — the dict-counter pattern, built in

```
from collections import Counter
Counter(["a", "b", "a", "c", "a"])     # Counter({'a': 3, 'b': 1, 'c': 1})
Counter("mississippi").most_common(1)  # [('s', 4)]  → the top item
```

Does Lesson 10's `counts.get(x, 0) + 1` loop for you. Learn the manual way first (to understand it), then reach for `Counter`. `.most_common(n)` gives the top n by count.

`enumerate()` — loop with a counter AND the item

```
for i, task in enumerate(tasks, start=1):   # i = 1,2,3...  task = each item
    print(f"{i}. {task}")
```

Cleaner than `for i in range(len(tasks)): tasks[i]`.

Inline `if` (ternary) — a one-line if/else that produces a value

```
mark = "x" if done else " "        # "x" if done is True, else " "
```

Handy for short choices; use a normal `if/else` for anything longer.

Digit extraction with `% 10` and `// 10`

```
n = 807
n % 10    # 7   → the LAST digit
n // 10   # 80  → chop the last digit off
# loop these to peel digits (units-first): 807 → [7, 0, 8]
```

Big-O intuition (how "fast" an approach is)

- **O(n)** = work grows *with* the input (one pass). Fast.
- **O(n²)** = nested loops, work grows with the *square*. Slow for big inputs.
- Swapping a list-scan (`x in list` , O(n)) for a set/dict lookup (O(1)) is the usual speed-up.

`if __name__ == "__main__":` — only auto-run when run directly

```python
def main():
    ...
if __name__ == "__main__":   # runs when you do `python3 file.py`,
    main()                    # but NOT when the file is imported elsewhere
```

Lets you import a file's functions (e.g. for testing) without its script auto-running.

LeetCode format note

Problems give you a **class + method** to fill in (`class Solution: def twoSum(self, nums, target):`). Put your logic inside the method; `nums` / `target` arrive as parameters; leave `self` as-is (it's an OOP thing — Lesson 13).

Design Principles

Not syntax — *habits* learned from real bugs in our projects. These matter as code grows.

- **Compute vs. report (pure functions).** A function that *calculates* something shouldn't also `print`. Why: you can't reuse a printing function for "what-if" simulation — it spams output. (Tic-Tac-Toe's `check_winner` had to become silent so the bot could use it to plan.)
- **One variable, one job.** Don't make a single variable mean two things. (The typing test's `minutes` needed to be *both* whole-minutes-for-display and float-minutes-for-rate → split into two.)
- **DRY — Don't Repeat Yourself.** Duplicated logic drifts out of sync (fix one copy, forget the mirror). Two near-identical blocks → extract a function. (Bit us: `remove_task` / `update_status`, and the two win-check blocks in Tic-Tac-Toe.)
- **Guard clauses.** Handle the bad/edge case first and `return` early, keeping the main logic un-indented.
- **Separate presentation from data.** Display tweaks work on a *copy*, never the real state. (Tic-Tac-Toe showed indices in empty cells via a `.copy()`, leaving the true board untouched.)
- **Validate untrusted input.** Wrap `int(input(...))` in `try` / `except`, range-check, reject bad values with a friendly message + `continue` — never let one typo crash the program.
- **Delete debug prints before "done."** A leftover `print(secret)` spoils the game. 😄

Style Conventions (PEP 8)

Not enforced by Python, but this is what idiomatic Python looks like:

Do	Don't
`name = "Raj"` (spaces around `=`)	`name="Raj"`
`print(...)` (no space before `(`)	`print (...)`
`first_name` (snake_case for variables)	`firstName`, `FirstName`
`if x > 5:` (no parens)	`if (x > 5):`
Capitalized names → for **classes** only (later)	naming a variable `Students`

⚠️ **Never name variables after built-ins:** `list`, `str`, `dict`, `sum`, `len`, `type`, `input`, `id`. Doing so **shadows** (overwrites) the built-in for the rest of the program. Use `my_list`, `numbers`, etc.

Common Error Types

Error	Caused by	Example
`SyntaxError`	Invalid code Python can't even parse	f-string quote clash, missing `:`
`ValueError`	Right type, wrong value	`int("hello")`
`TypeError`	Wrong type for the operation	`"2" + 2`
`IndexError`	List index out of range	`[1,2,3][99]`
`KeyError`	Dictionary key doesn't exist	`person["missing"]`
`ZeroDivisionError`	Divide by zero	`10 / 0`, `average([])`
`NameError`	Using a name that doesn't exist	typo in a variable name

Concepts covered: variables · data types · operators · strings & f-strings · type conversion · `input()` · conditionals · loops (`for` / `while` / `while True`) · `break` / `continue` / `pass` · lists · `.pop()` vs `.remove()` · aliasing & copying · dictionaries · functions · `return` vs `print` · guard clauses · `None` · `try` / `except` · slicing · string methods · validation methods (`.isdigit()` …) · immutability · sets · dict-counter · hash-map lookup · list comprehensions · tuples & unpacking · multiple return values · `import` / `random` / `time` · `Counter` · `enumerate` · built-ins (`min` / `max` / `sum` / `sorted` / `all` / `any`) · ternary · `__main__` · Big-O intuition · design principles.

Projects built: see `Exercises.md` for the full list with concept tags — quiz, to-do apps, number guesser, RPS, hangman, wordle, dice stats, password checker, movie analyzer, word-frequency counter, Mastermind (+ solver hints), typing-speed test, Tic-Tac-Toe (+ rule-based AI), Two Sum (LeetCode).

Next up: Lesson 13 — Classes / OOP (the big one: unlocks linked lists + "Add Two Numbers" AND every ML framework) → then linked lists → then Phase 1 of the AI roadmap (NumPy & pandas — `movies.py` was a manual preview of pandas). See `AI_ROADMAP.md`.